University of Eloued

Faculty of Exact Sciences

Department of Computer science

For Second-year Master2: IA & Data Science

Rapport de TP04: Containerization and Big Data Environment Deployment Using Docker

Étudiante : Akila Saker

Module : Big Data Analytics

Année universitaire : 2026–2027

1. Introduction

Docker is a containerization platform that allows applications and their dependencies to be packaged into lightweight, portable, and isolated environments called containers.

The purpose of this practical work is to understand Docker fundamentals and apply containerization techniques to deploy a Big Data analysis application. The project uses a fire monitoring dataset previously prepared in TP03.

The application is developed in Python using the Pandas library to analyze sensor measurements and identify fire alarm events.

2. Objectives

The main objectives of this practical work are:

Understand Docker images and containers.

Create a custom Docker image for a Python data analysis application.

Build and run the Docker image locally.

Publish the image on Docker Hub.

Pull the published image from Docker Hub.

Create and configure a custom Docker bridge network.

Connect multiple containers to the same network.

Use Docker Compose to automate application deployment.

Analyze a dataset inside a Docker container.

3. Project Structure

The project directory is organized as follows:

```
TP04_Docker_BigData/
|
├── data/
|
└── tp03_fire_monitoring_sample.csv
|
├── screenshots/
```

```
|
├── app.py
├── requirements.txt
├── Dockerfile
├── compose.yaml
└── README.md
```

Description of files

•**app.py:** Contains the Python code used to analyze the fire monitoring dataset.

requirements.txt: Specifies the required Python dependencies.

Dockerfile: Defines the instructions for building the Docker image.

compose.yaml: Defines the application service and its volume configuration.

data/: Contains the CSV dataset.

screenshots/: Stores screenshots documenting the practical work.

README.md: Provides project documentation and execution instructions.

4. Dataset Description

The project uses the fire monitoring dataset created during TP03.

The dataset contains 10,000 records and 7 columns:

timestamp

sensor_id

temperature

humidity

smoke_level

fire_alarm

sensor_status

The dataset represents measurements collected from environmental sensors to monitor possible fire conditions.

The `fire_alarm` column indicates whether a fire alarm has been triggered:

0: Normal condition.

1: Fire alarm detected.

5. Python Application

The application was developed using Python and Pandas.

Its main operations include:

Checking whether the dataset exists.

Loading the CSV file.

Displaying the dataset dimensions.

Displaying the column names.Checking missing values.

Calculating the distribution of fire alarm events.

Computing the average temperature, humidity, and smoke level.

The application uses the following dataset path inside the container:

```
DATA_PATH = "/app/data/tp03_fire_monitoring_sample.csv"
```

6. Creating the Docker Image

A Dockerfile was created to define the application environment.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

CMD ["python", "app.py"]
```

The image was built using:

```
docker build --pull -t tp04-fire-analysis:1.0 .
```

The image was successfully created.

7. Publishing the Image on Docker Hub

The local image was tagged with the Docker Hub repository name:

```
docker tag tp04-fire-analysis:1.0 akilasaker/tp04-fire-analysis:1.0
```

Then, it was uploaded:

```
docker push akilasaker/tp04-fire-analysis:1.0
```

The image was also pulled successfully:

```
docker pull akilasaker/tp04-fire-analysis:1.0
```

Docker Hub repository:

https://hub.docker.com/r/akilasaker/tp04-fire-analysis

This step demonstrates that the image can be shared and retrieved from a remote registry.

8. Running the Docker Container

The application was executed inside a Docker container using:

`docker run --rm -v "${PWD}/data:/app/data:ro" tp04-fire-analysis:1.0`The volume option mounts the local dataset directory into the container.

The `:ro` option means that the dataset is mounted in read-only mode.

This approach keeps the dataset outside the Docker image while making it accessible to the application.

The container executed the Python analysis successfully.

9. Docker Network Configuration

A custom bridge network named `tp04-network` was created:

```
docker network create --driver bridge tp04-network
```

The network was verified using:

```
docker network ls
```

The network inspection command was:

```
docker network inspect tp04-network
```

The network configuration showed:

Network name: `tp04-network`

Driver: `bridge`

Scope: `local`

Subnet: `172.19.0.0/16`

Gateway: `172.19.0.1`

Three containers were created to demonstrate the network configuration:

tp04-node1

tp04-node2

tp04-node3

Each container was configured to use the custom Docker network.

This demonstrates how multiple containers can be connected to the same isolated bridge network.

10. Docker Compose Deployment

Docker Compose was used to simplify application deployment and configuration.

The `compose.yaml` file contains:

```
services:

data-analysis:

build: .

container_name: tp04-analysisvolumes:

- ./data:/app/data:ro

restart: "no"
```

The application was launched with:

```
docker compose up --build
```

Docker Compose automatically built the image when necessary, created the container, mounted the dataset, and executed the analysis.

The application completed successfully.

The following command was used to stop and remove the Compose resources:

```
docker compose down
```

11. Results and Analysis

The execution of the application produced the following results:

Metric	Result
Number of records	10,000
Number of columns	7
Missing values	0
Normal conditions (fire_alarm = 0)	9,250
Fire alarms (fire_alarm = 1)	750
Average temperature	29.93
Average humidity	

52.69

Average smoke level

100.50

Interpretation

The dataset contains 9,250 normal records and 750 fire alarm records.

No missing values were detected in the dataset.

The average values were successfully calculated for temperature, humidity, and smoke level.

The results confirm that the Python application can access and process the dataset inside the Docker environment.

12. Technical Challenges and Solutions

During the practical work, several technical points were addressed:

1. Dataset access inside the container

Initially, the application could not find the CSV file because the dataset was stored outside the image.

Solution: A bind mount was configured to make the local data directory accessible inside the container.**2. Docker image distribution**

The locally built image needed to be published and retrieved from Docker Hub.

Solution: The image was tagged, pushed, and pulled successfully.

3. Container networking

A custom bridge network was required to organize communication between multiple containers.

Solution: The `tp04-network` network was created and inspected using Docker commands.

4. Application deployment

Running the application manually required several commands.

Solution: Docker Compose was used to automate the build, container creation, and volume configuration.

13. Conclusion

This practical work provided hands-on experience with Docker containerization and deployment.

A Python-based fire monitoring analysis application was successfully packaged into a custom Docker image and published on Docker Hub.

The application was executed inside a container using a read-only bind mount to access the CSV dataset. A custom bridge network was also created to demonstrate multi-container networking.

Finally, Docker Compose was used to simplify the deployment process.

Overall, this practical work demonstrated how Docker can provide a portable, reproducible, and isolated environment for data analysis applications and Big Data

screenshots

```
alysis   | ==================================================

Dataset shape: (10000, 7)

Columns:
['timestamp', 'sensor_id', 'temperature', 'humidity', 'smoke_level', 'fire_alarm', 'sensor_status']

Missing values:
timestamp        0
sensor_id        0
temperature      0
humidity         0
smoke_level      0
fire_alarm       0
sensor_status    0
dtype: int64

Fire alarm distribution:
fire_alarm
0    9250
1     750
Name: count, dtype: int64

Mean temperature: 29.93

Mean humidity: 52.69

Mean smoke_level: 100.50

Analysis completed successfully.
```

compose_analysis


```
        ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
        "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
        "com.docker.network.enable_ipv4": "true",
        "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
        "04ad295b15399d7803204cd474b0d36402588f4ae81d1e8b7124f21926bc3f85": {
            "Name": "tp04-node3",
            "EndpointID": "6c3e31d772cb9de1a4f95f5ee05a059b6ce6922daeca64fe9b700465662cd19e",
            "MacAddress": "e2:7d:6c:53:58:92",
            "IPv4Address": "172.19.0.4/16",
            "IPv6Address": ""
        },
        "1defb90a7e4adbf342cfcda31f47ea9ad2f382d0b848f6744655dde766939d05": {
            "Name": "tp04-node2",
            "EndpointID": "3f2bd6d0c929fba11d53cc2412d4a3a450bceaae8a58b9b298e5117ff3b4b471",
            "MacAddress": "1a:b0:12:4d:d5:68",
            "IPv4Address": "172.19.0.3/16",
            "IPv6Address": ""
        },
        "99fd5841b1447d80e2bcd7dfacb872b573a56026c109c4b7fba9f26846cb86cd": {
            "Name": "tp04-node1",
            "EndpointID": "bb223c62f062921d7c295ba5161ae3ff5bed09bdef645fd791b7326efcd3b64a",
            "MacAddress": "76:5b:80:02:7c:80",
            "IPv4Address": "172.19.0.2/16",
            "IPv6Address": ""
        }
    },
    "Status": {
        "IPAM": {
            "Subnets": {
                "172.19.0.0/16": {
                    "IPsInUse": 6,
                    "DynamicIPsAvailable": 65530
                }
            }
        }
    }
}
```


04_docker_containers_network

```
Digest: sha256:026a863667ffc1962db370423120206ba04887690d87cf4a233d92fd44239ce3
Status: Image is up to date for akilasaker/tp04-fire-analysis:1.0
docker.io/akilasaker/tp04-fire-analysis:1.0
```

03_dockerhub_pull

```
==================================================

Dataset shape: (10000, 7)

Columns:
['timestamp', 'sensor_id', 'temperature', 'humidity', 'smoke_level', 'fire_alarm', 'sensor_status']

Missing values:
timestamp        0
sensor_id        0
temperature      0
humidity         0
smoke_level      0
fire_alarm       0
sensor_status    0
dtype: int64

Fire alarm distribution:
fire_alarm
0    9250
1     750
Name: count, dtype: int64

Mean temperature: 29.93

Mean humidity: 52.69

Mean smoke_level: 100.50
```

02_data_analysis.png